

Machine Learning for Finance — Research Paper

Zaixuan Quan
s86zquan@uni-bonn.de

July 22, 2026

Contents

1	Introduction	2
2	Data	2
2.1	Data Source	2
2.2	Sample and Universe	3
2.3	Summary Statistics	4
3	Methodology	5
3.1	General Approach: Rolling Window + Pooling Training	5
3.2	Models	5
3.3	Backtest Design	5
3.4	Hyperparameter Tuning	5
4	Results	6
4.1	Model Comparison	6
4.2	Risk and Drawdown Analysis	9
5	Critical Reflection	11
5.1	Classification Model Choice	11
5.2	Filtering Micro-Cap Stocks	11
5.3	Forward-Looking Bias	11
5.4	Predict well $\neq$ Trade well	12
5.5	Potential Problems in Tuning Hyperparameters	12
5.6	Why Dimension Reduction Did Not Work as Well as Expected	12
5.7	Ignoring Transaction Costs and Slippage	12
5.8	Not Enough Features	13
6	Conclusion	13

1 Introduction

In this paper, we study the problem of predicting the monthly stock returns. We use CRSP monthly stock file to construct the data set and the goal is to predict the cross-sectional stock returns of all US equities from 1972 to 2025. The basic idea is to first implement different machine learning models to predict the monthly stock returns and then carry out long-short trading strategies.

As for the models, we choose four groups of models including linear models (OLS, Ridge, Lasso), PCA and PLS, tree models (decision tree, Random Forest, bagging, boosting), and classifier models (logistic regression and multinomial logistic regression). To train the models, we choose MSE as the loss function/optimization target. In addition, we use the first 240 months as the training set and the following 60 months as the validation set to tune the hyperparameters. This way of splitting the whole dataset ensures that we do not leak any future information when choosing the parameters. Then we use a fixed-length rolling window prediction method to test our models and implement the corresponding strategies. For the backtest part, we compare different models' cumulative returns, Sharpe ratios, volatility, and maximum drawdowns.

Overall, we find that the boosting model performs best, which is not a surprising result. Besides, we find that after 2018, the cross-sectional strategy cannot work as well as before, which means this is not only about training models but also about the fact that the whole market structure has changed. We also compare large-cap stocks versus all stocks, and find that most of the excess return comes from micro-cap stocks. This is consistent with the fact that micro-cap stocks are less liquid and harder to trade. Although this research project's prediction power is not that strong, it is still interesting to see how machine learning methods help us with the practical prediction problem and to see how the stock market has changed over the past 20 years.

The rest of the paper is organized as follows: Section 2 is about the data collection and preprocessing, Section 3 is about the model, Section 4 is about the results, Section 5 is a discussion and reflection on the difficulties and potential drawbacks of the research project, and Section 6 is the conclusion.

2 Data

2.1 Data Source

The main data source is the **CRSP monthly stock file**, covering all US equities from January 1972 to December 2025. The fields it provides fall into two groups:

- **Stock identification:** permno code, primary exchange, share type, industry/sector classification
- **Time-series data:** monthly return, monthly price, monthly market capitalization, monthly trading volume, shares outstanding

With this dataset, we can construct some features such as **size** (log market capitalization), **mom1m** (last month's return), **turnover** (monthly volume / shares outstanding).

The second data source is **OSAP (Open Source Asset Pricing Library)**, which provides us with a variety of factors used in asset pricing. We select the following factors:

- **Value:** BM (book-to-market), EP (earnings-to-price)

- **Profitability / Quality:** `GP` (gross profitability), `roaq` (quarterly ROA), `Accruals` (Sloan 1996 accruals), `PS` (Piotroski F-score)
- **Growth / Investment:** `AssetGrowth` (annual asset growth), `ShareIss1Y` (1-year share issuance)
- **Momentum / Reversal:** `Mom12m` (12-month momentum), `Lrreversal` (long-run reversal), `IndMom` (industry momentum), `High52` (52-week high ratio)
- **Risk / Volatility:** `Beta` (rolling CAPM beta), `RealizedVol` (realized volatility), `IdioVol3F` (idiosyncratic volatility), `Leverage` (liabilities/market equity)
- **Liquidity:** `Illiquidity` (Amihud illiquidity)
- **Earnings Surprise:** `EarningsSurprise` (standardized unexpected earnings)
- **R&D:** `RD` (R&D intensity)

We also obtain macro data from **Goyal-Welch Predictor Data**. It contains the following variables:

- `tbl`: treasury bill rate
- `tms`: term structure spread
- `dfy`: default spread(BAA-AAA)
- `Rfree`: risk free rate

2.2 Sample and Universe

Here we also need to construct subsamples of the whole dataset. Since the dataset contains all the stocks in the US market, the capitalization varies a lot. Some micro-capitalized stocks can bring extremely high returns but with low liquidity, which reduces the feasibility of our trading strategy. To minimize the micro-capitalized stocks' influence, we filter out some micro-cap stocks according to HXZ (2020).¹ In that setting, they put the threshold at the 20th percentile of stocks' market capitalization in NYSE. Any stock from any exchange whose market cap falls below that cutoff is classified as a "microcap" and dropped. Since this method drops out around 60 percent of stocks in the CRSP universe, we implement a milder filtering cutoff: we drop stocks that fall in the bottom 10 percent of stocks ranked by market capitalization each month.

This split is helpful for us to identify the role of micro-cap stocks in the stock market and their limitations. We will revisit this part in Section 4.

¹Hou, K., Xue, C., and Zhang, L. (2020). Replicating Anomalies. *The Review of Financial Studies*, 33(5), 2019–2133.

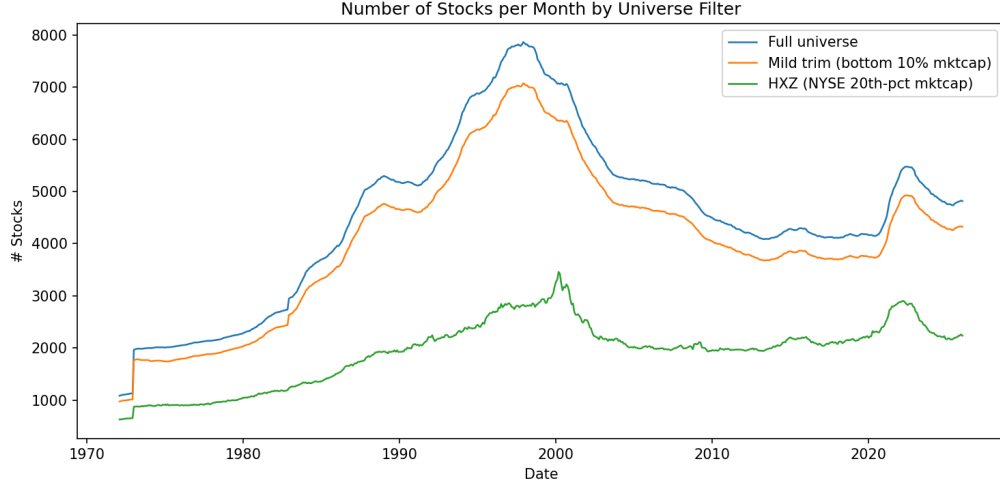


Figure 1: Number of stocks per month under the three universe definitions: full universe, mild trim (bottom 10% by market cap dropped each month), and HXZ microcap exclusion (below NYSE 20th-percentile market cap dropped each month).

2.3 Summary Statistics

Table 1: Summary statistics of the target variable and features (pooled panel, all permno-month rows, after cross-sectional median imputation)

	N	mean	std	min	P5	P25	Median	P75	P95	max
y	2,922,274	0.0077	0.1970	-1.0066	-0.2367	-0.0723	-0.0028	0.0693	0.2659	38.9960
size	2,950,056	12.1286	2.2656	2.3571	8.7135	10.4679	11.9611	13.6435	16.1086	22.3169
mom1m	2,948,972	0.0111	0.1950	-0.9957	-0.2308	-0.0680	0.0000	0.0715	0.2672	39.0000
price	2,950,056	49.4722	3,053.5	0.0062	1.0200	5.0700	13.0800	26.8750	69.1100	800,540.0
turnover	2,950,056	210.0692	12,255.3	0.0000	5.5325	22.5018	60.8111	146.6255	444.5400	11,922,570.0
mom12m	2,892,716	0.1273	0.7951	-1.0000	-0.6250	-0.1988	0.0467	0.2997	1.0600	436.6845
high52	2,892,716	0.7385	0.3691	0.0005	0.2574	0.5704	0.7786	0.9231	1.0631	262.3832
ret_vol	2,892,716	0.0344	0.0335	0.0000	0.0085	0.0166	0.0261	0.0418	0.0859	8.4777
idio_vol	2,892,716	0.0296	0.0308	0.0000	0.0067	0.0133	0.0218	0.0361	0.0767	7.7997
ind_mom	2,892,716	0.0909	0.1702	-0.9136	-0.1587	-0.0018	0.0830	0.1694	0.3563	10.5068
beta	2,892,716	1.0598	0.7485	-17.8663	0.1410	0.6298	0.9557	1.3473	2.3670	52.6319
share_issuance	2,892,531	0.9390	654.7040	-0.9982	-0.0528	0.0000	0.0063	0.0291	0.3715	707,720.6
amihud_illiq	2,892,716	0.0000	0.0001	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0250
leverage	2,892,716	2.4611	16.8695	0.0000	0.0431	0.2590	0.6142	1.5337	8.9588	5,277.2
tbl	2,950,056	0.0406	0.0300	0.0001	0.0004	0.0148	0.0444	0.0559	0.0896	0.1630
tms	2,950,056	0.0190	0.0148	-0.0365	-0.0063	0.0087	0.0189	0.0313	0.0411	0.0455
dfy	2,950,056	0.0100	0.0040	0.0055	0.0060	0.0071	0.0090	0.0116	0.0171	0.0338
mom12m_tbl	2,892,716	0.0053	0.0376	-0.1248	-0.0284	-0.0051	0.0002	0.0091	0.0484	24.8473
mom12m_tms	2,892,716	0.0030	0.0193	-0.5460	-0.0146	-0.0029	0.0004	0.0055	0.0249	4.2838
mom12m_dfy	2,892,716	0.0010	0.0075	-0.0336	-0.0067	-0.0019	0.0004	0.0027	0.0104	3.0131
beta_tbl	2,892,716	0.0420	0.0448	-0.6690	0.0002	0.0061	0.0322	0.0622	0.1246	2.6105
beta_tms	2,892,716	0.0202	0.0235	-0.4520	-0.0067	0.0053	0.0159	0.0305	0.0619	1.2775
beta_dfy	2,892,716	0.0106	0.0086	-0.2501	0.0012	0.0054	0.0089	0.0138	0.0261	0.6424
bm	2,892,716	-0.7968	0.9231	-10.1310	-2.4669	-1.1336	-0.7147	-0.2690	0.4801	6.8214
gross_profit	2,950,056	0.3221	0.3049	-25.2083	0.0000	0.2027	0.3153	0.4030	0.7703	6.1928
roa_q	2,892,705	-0.0049	0.2726	-58.0917	-0.1034	0.0008	0.0067	0.0149	0.0410	171.9972
asset_growth	2,950,056	0.1743	1.9873	-1.0000	-0.2247	0.0050	0.0768	0.1474	0.6792	679.3918
accruals	2,950,056	-0.0305	0.1088	-24.3138	-0.1607	-0.0572	-0.0290	-0.0026	0.1020	6.2160
mom36m	2,892,716	0.2970	1.2579	-1.0000	-0.6908	-0.1733	0.1496	0.4734	1.6743	544.3116

Note: y is the forward monthly excess return. A small number of rows (< 0.01%) with infinite values in `share_issuance` and `roa_q`, arising from division by a near-zero denominator in their construction, are excluded from this table's mean/std/max only.

3 Methodology

3.1 General Approach: Rolling Window + Pooling Training

Since stock prices are time series data, we will use rolling window training and testing. First we use the first 300 months(1972-01 to 1996-12) to tune hyperparameters. Then we use the remaining data to train the model and test. The length of window is 240 months.

We do **pooling training**, which means we train our model on all the observations in the window rather than training a separate model for each stock. We use the same model to predict next month's returns for all stocks. In short, we do not have specific models for each stock. Pooling can be used to maintain a large training sample size.

3.2 Models

We choose basic machine learning models to predict monthly stock returns, excluding deep learning models. We train the models on a rolling window of 240 months and optimize MSE for each model except the classification models. For the classification models, we use cross-entropy as the metric to optimize.

- **linear models** linear regression, ridge regression, lasso regression
- **dimensionality reduction models** PCA, PLS
- **tree models** decision tree, random forest, bagging, boosting
- **classification models** logistic regression, multinomial logistic regression

3.3 Backtest Design

For the i -th month, we train the model on the window from $i - 240$ to $i - 1$ and then predict the i -th month's returns $\hat{y}_{i,t}$ for each stock. We then sort the stocks by their predicted returns into 10 deciles and construct portfolios from the first and last deciles in two ways: equal-weighted and market-cap-weighted. Market-cap weighting reduces noise from micro-cap stocks and gives cleaner results, because the weight of micro-cap stocks is considerably lower than under equal weighting. We long (buy) the first decile and short (sell) the last decile. This gives us our strategy's return for every month, from which we compute evaluation metrics including the cumulative return, Sharpe ratio, volatility, and maximum drawdown.

3.4 Hyperparameter Tuning

For linear models and decision trees, tuning is based on the first 240 months (1972-01 to 1991-12) as the training set and the following 60 months (1992-01 to 1996-12) as the validation set. We tune λ for Ridge/Lasso and the **number of components** for PCA and PLS. We tune not only by MSE but also by sign accuracy, the fraction of correct sign predictions for all stocks in the whole validation set. Since low MSE does not necessarily mean we can earn positive returns, we also consider sign accuracy. The selection rule picks, among the three candidates with the lowest MSE, the one with the highest sign accuracy.

For Lasso specifically, we need one more step. At sufficiently strong regularization, Lasso can shrink every coefficient to zero, producing an intercept-only prediction for every stock in a given month, so we exclude candidates with all-zero coefficients. To do this, we calculate the variance of the predictions for every month and drop those candidates with zero variance.

For the ensemble tree models, we tune `n_estimators` and leave `max_depth` at its default value, because our laptop is not powerful enough to tune every hyperparameter. We use 240 months for training and validation together, so we do not need an additional validation set. Here we choose 1977.01 to 1996.12 as our training set to keep the same test period as before. For Random Forest and Bagging, we get the validation set for free because of bootstrapping. For both models, `n_estimators` is chosen from the grid $\{25, 50, 100, 200\}$ using the OOB R^2 score, but instead of taking the value with the single highest OOB score, we select the smallest `n_estimators` whose OOB score is within 5% of the best score in the grid. OOB performance typically plateaus well before the largest grid value, so this rule achieves essentially the same validation performance at a much lower cost.

Boosting (`HistGradientBoostingRegressor`) is tuned differently: rather than a manual grid search, it uses built-in early stopping, with an internal validation split (`validation_fraction=0.1`) and training stopped once the validation score fails to improve for 10 consecutive rounds (`n_iter_no_change=10`).

4 Results

4.1 Model Comparison

To keep the comparison meaningful, we focus on the two filtered universes (mild trim and HXZ microcap exclusion) rather than the unfiltered full universe: the full universe is dominated by extremely small, illiquid stocks whose return swings are large enough to overwhelm any comparison across models. For the nine models (eight regressors plus Multinomial Logit), Table 2 and Table 3 report the annualized Sharpe ratio, total compounded return (e.g. 1.00 = +100%), and annualized volatility of the long-short decile portfolio, computed both equal-weighted and market-cap-weighted, under the mild and HXZ universes respectively.

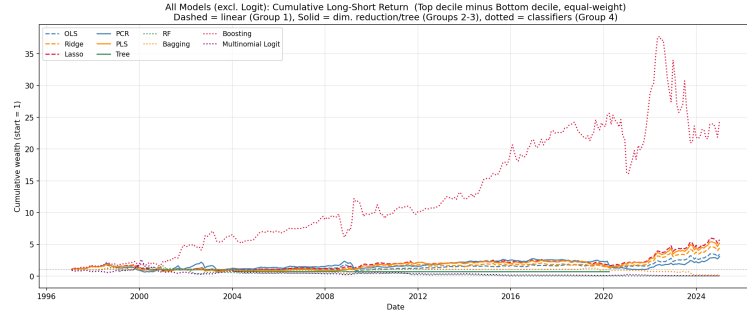
Table 2: Performance under the mild universe filter (bottom 10% of stocks by market cap excluded each month)

Model	Equal-weight			Cap-weight		
	Sharpe	Return	Vol	Sharpe	Return	Vol
OLS	0.319	2.45	0.228	0.048	-0.31	0.217
Ridge	0.362	3.43	0.213	0.073	-0.17	0.211
Lasso	0.405	4.73	0.215	0.153	0.33	0.212
PCA	0.294	2.07	0.274	-0.128	-0.82	0.244
PLS	0.382	4.19	0.229	0.131	0.12	0.226
Decision Tree	-0.469	-0.31	0.170	-0.427	-0.31	0.181
Random Forest	-0.316	-0.99	0.386	-0.269	-0.99	0.428
Bagging	-0.244	-0.83	0.256	-0.419	-0.94	0.272
Boosting	0.584	23.43	0.246	0.255	1.53	0.275
Multinomial Logit	-0.134	-0.95	0.349	0.006	-0.82	0.358

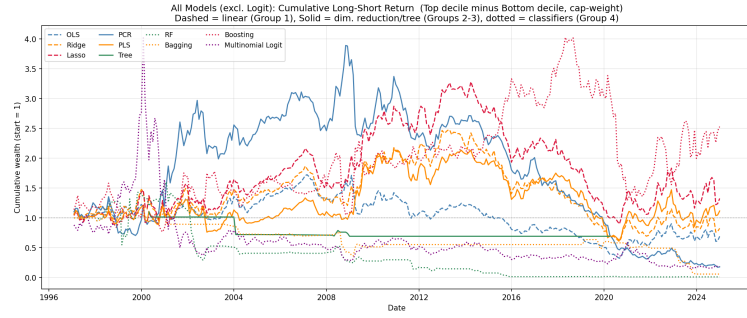
Table 3: Performance under the HXZ universe filter (stocks below the NYSE 20th-percentile market cap excluded each month)

Model	Equal-weight			Cap-weight		
	Sharpe	Return	Vol	Sharpe	Return	Vol
OLS	0.045	-0.22	0.185	-0.001	-0.41	0.192
Ridge	-0.090	-0.59	0.179	-0.137	-0.70	0.187
Lasso	-0.102	-0.63	0.180	-0.133	-0.72	0.195
PCA	-0.065	-0.84	0.300	-0.168	-0.93	0.300
PLS	-0.020	-0.43	0.180	-0.068	-0.57	0.186
Decision Tree	0.160	0.09	0.237	-0.058	-0.25	0.273
Random Forest	-0.191	-0.84	0.265	-0.028	-0.76	0.332
Bagging	-0.111	-0.45	0.224	-0.191	-0.63	0.263
Boosting	0.105	-0.06	0.240	-0.019	-0.61	0.241
Multinomial Logit	-0.086	-0.90	0.328	0.027	-0.73	0.337

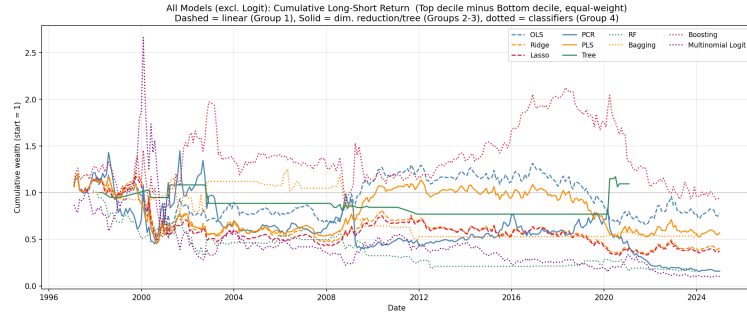
Figure 2 plots the cumulative long-short return for all models under each of the four combinations of universe filter and weighting scheme.



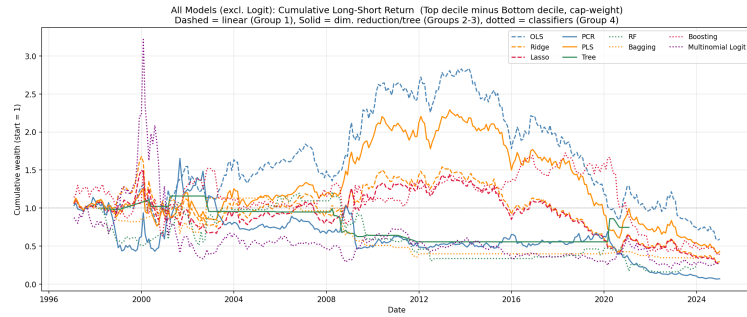
(a) Mild filter, equal-weight



(b) Mild filter, cap-weight



(c) HXZ filter, equal-weight



(d) HXZ filter, cap-weight

Figure 2: Cumulative long-short return (top decile minus bottom decile, starting wealth = 1) across universe filters and weighting schemes. Binary Logit is excluded from the legend in all four panels.

Two patterns are consistent across both filtered universes. First, moving from equal weighting to cap weighting reduces or eliminates the performance of almost every model: cap weighting puts more weight on large, liquid stocks and proportionally less on the smaller stocks each decile portfolio contains, so a model whose Sharpe ratio survives this change is relying less on small-

stock mispricing. Second, the HXZ filter removes most of the remaining edge that the mild filter still allowed: only three of the nine models keep a positive equal-weighted Sharpe ratio under HXZ, and only two keep a positive Sharpe ratio once cap-weighted as well.

Looking at all four Sharpe ratios together (mild/HXZ $\times$ equal/cap-weight) for each model gives a rough robustness ranking:

- **Most robust – OLS and Boosting.** Both stay positive in three of the four combinations, and the one negative case for each (cap-weighted HXZ) is close to zero (-0.001 for OLS, -0.019 for Boosting) rather than a real reversal. Boosting’s edge is much larger to start with (Sharpe 0.584 under the mild, equal-weighted setting) but also shrinks the most, by about 80%, once the universe is tightened and weighting is changed; OLS starts smaller (Sharpe 0.319) but decays more gently.
- **Microcap-dependent – Ridge, Lasso, PLS, PCA.** All four post a positive Sharpe ratio under the mild filter regardless of weighting (roughly 0.29 to 0.41 equal-weighted), but every one of them turns negative under the stricter HXZ filter, equal- or cap-weighted alike. The mild-filter edge and the HXZ-filter loss appear together consistently enough that it looks like the same small-cap exposure driving both.
- **Unstable – Decision Tree.** The single tree is the worst model under the mild filter (Sharpe -0.469 equal-weighted) but the best equal-weighted model under HXZ (Sharpe 0.160), then negative again once cap-weighted. A model whose ranking flips from last to first between two similar universe definitions is not displaying a real signal, just high variance from refitting a shallow tree every month.
- **Consistently weak – Random Forest, Bagging, Multinomial Logit.** All three stay negative, or statistically indistinguishable from zero, across nearly all four combinations. Multinomial Logit’s classification accuracy (around 27%) does beat the 20% random-guess baseline for a 5-class split, but that classification edge never shows up as a profitable long-short return once microcaps are filtered out or down-weighted.

In short, only OLS and Boosting hold up once the universe is tightened and positions are cap-weighted; for every other model, the apparent edge either depends on small-cap stocks the stricter filter removes, or was never reliably positive to begin with. The very large cumulative returns reported under looser, equal-weighted settings should therefore not be read as representative of what an investor could plausibly capture.

4.2 Risk and Drawdown Analysis

Table 4 reports, for each model, the single worst month-end drawdown and when it occurred, under the mild and HXZ filters (equal-weighted portfolios).

Table 4: Worst single-month drawdown and its date, by model and universe filter

Model	Mild filter		HXZ filter	
	Date	Drawdown	Date	Drawdown
OLS	2000-12	-0.59	2000-08	-0.61
Ridge	2003-04	-0.56	2020-08	-0.72
Lasso	2001-01	-0.55	2020-08	-0.73
PCA	2000-03	-0.69	2024-01	-0.89
PLS	2003-04	-0.62	2000-08	-0.62
Decision Tree	2020-03	-0.36	2020-02	-0.29
Random Forest	2023-10	-0.99	2023-11	-0.85
Bagging	2024-12	-0.87	2011-11	-0.61
Boosting	2023-09	-0.44	2000-11	-0.64
Multinomial Logit	2024-08	-0.98	2024-07	-0.96

Two models give a clean, filter-invariant signal: **Random Forest** and **Multinomial Logit** both have their single worst drawdown in 2023-2024 under *both* the mild and the HXZ filter. **OLS** gives the opposite, equally clean signal: its worst drawdown is anchored to the 2000 dot-com crash under both filters. The other models are more filter-dependent – Ridge and Lasso move from an early-2000s worst case under the mild filter to 2020 (the COVID crash) under HXZ, and PCA and Boosting flip between an early-2000s and a 2020s worst case depending on the filter – so for these models drawdown timing should be read as sensitive to exactly how the universe is constructed, not as a stable fact about the model. The Decision Tree is a special case: many of its worst months share an *identical* drawdown value (e.g. -0.363 repeated across several non-adjacent dates under the mild filter), which is a sign that the shallow single tree produces only a handful of distinct leaf predictions and repeatedly assigns the same decile sort on unrelated dates, rather than genuinely reacting to market conditions. This is consistent with the Decision Tree being flagged as the least reliable model in Table 2 and Table 3.

The recent-year concentration of losses for Random Forest and Multinomial Logit, together with the broader pattern from Section 4 that almost every model’s edge shrinks or disappears once equal weighting is replaced by cap weighting, points to the same underlying story: U.S. equity returns over the last several years have been increasingly concentrated in a small number of large, mega-cap stocks, while this project’s long-short strategy is built on equal-weighted decile portfolios that lean on dispersion among smaller, more numerous stocks. As that concentration increased, the small-cap mispricing this strategy depends on became both a shrinking and a riskier source of return, which shows up here as worse tail risk in the recent part of the sample for the models whose edge was most microcap-dependent to begin with.

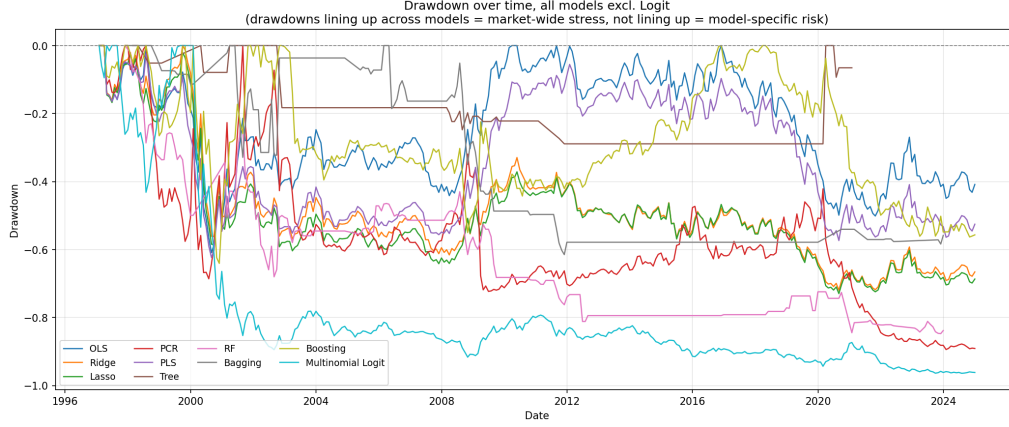


Figure 3: Drawdown over time, all models, HXZ-filtered universe, equal-weighted. Lining up across models points to market-wide stress; not lining up points to model-specific risk.

5 Critical Reflection

5.1 Classification Model Choice

At first, we chose Logistic Regression as our classification model. However, we found that the model brought an extremely high level of cumulative return. That is partly because the logit model's target is binary, indicating whether the stock's return exceeds next period's cross-sectional median return or not. We then long the stocks predicted to be above the median return and short the stocks predicted to be below it. This method is too rough and does not suit our project well. What's more, it is much more easily influenced by extreme returns from a small number of stocks. We therefore turned to Multinomial Logistic Regression. In Multinomial Logistic Regression, we set the predicted variable to be $\{1, 2, 3, 4, 5\}$, and long the portfolio predicted to be 1 and short the portfolio predicted to be 5. This method works better for our project than the previous logit model.

5.2 Filtering Micro-Cap Stocks

As shown before, the baseline model, which contains all the stocks, brings us quite different results between equal-weighted and market-value-weighted portfolios; we therefore decide to take capitalization into account. Micro-cap stocks always bear more noise than large-cap stocks and are more exposed to potential insider trading. It is therefore a sensible choice for us to filter out these micro-cap stocks. In fact, a market-value-weighted portfolio not only accounts for market capitalization but also reduces the influence of certain small-cap stocks.

5.3 Forward-Looking Bias

Forward-looking bias is not a trivial problem for stock return predictions, because stock return data are time-series data. The nature of time-series data requires us to be careful not to leak future information into the model. When we first trained Random Forest, Bagging, and Boosting, we mistakenly used the most recent training window (the one closest to the end of the sample) to select the optimal hyperparameters and then applied that same choice across

the whole backtest. This was a serious mistake, because it means we used future data to make decisions for earlier test periods, which tends to overestimate model performance.

But we still have some forward-looking bias in choosing the hyperparameters for the ensemble tree models. Here we choose the 240 months from 1977.01 to 1996.12 as the training set and use the OOB score to select the best `n_estimators`. Although we use pooling training, meaning we use all the samples in the window together, we still have a potential problem: because the bootstrap resampling does not respect the time order within the window, a bootstrap sample used to fit the model can include observations from near the end of the window (e.g. 1996) while the corresponding out-of-bag validation observation comes from earlier in the window (e.g. 1977), so the OOB score can implicitly draw on later information to validate an earlier prediction. If we instead restricted each OOB fold to use only strictly earlier data, we would lose the efficiency advantage of bootstrapping.

5.4 Predict well $\neq$ Trade well

The goal of training is to predict next period's return and minimize mean squared error. But in practice, we also have to consider the sign of return, which means that we can only make profits when the model predicts correct direction. Although when choosing hyperparameters we consider both, for the rest part we only consider the magnitude of return. This is somehow instruct drawback of our methods. We can solve this problem by training models to give us best portfolio weights. This is an end-to-end learning approach, which can be implemented in Neural Networks.

5.5 Potential Problems in Tuning Hyperparameters

Because of the limitation of the laptop, we cannot implement the grid search in such a large parameter space. As a result, the hyperparameters we select are likely not fully optimal, and the reported performance may be somewhat conservative compared with what a more exhaustive, fully re-tuned search could achieve. With more computational resources, we would use a finer grid, parallelize the search, and re-tune hyperparameters.

5.6 Why Dimension Reduction Did Not Work as Well as Expected

Technically, dimension reduction can help reduce overfitting, but it does not work well in our project. The reason is that we only have 26 features but millions of observations. Dimension reduction is more helpful when there are relatively few observations and many features; in our setting, with far more observations than features, overfitting is less of a concern to begin with, so dimension reduction has less to offer.

5.7 Ignoring Transaction Costs and Slippage

All the strategies are based on the assumption that we can ignore transaction costs and slippage. When calculating the profits, we just use the closing price of every month and do not consider the bid-ask spread, which will overestimate the profits. Besides, we assume we can always buy the stock at the quoted price, but in practice this is not always true, namely the slippage problem. This usually happens when we detect a signal but cannot immediately buy or sell the stocks.

5.8 Not Enough Features

In this project, we use a range of features about the stock itself, including value, momentum, profitability, growth, risk, and liquidity, among others. These features are more like cross-sectional features, and we do not use many time-series features. Especially over the past ten years, we find that cross-sectional strategies are not as powerful and effective as before, so we need to add more time-series features. Besides, we should also add some features about the correlation between different stocks; for example, an increase in one stock's return can predict another stock's return. The lead-lag relationship between two stocks cannot be ignored. Sentiment features are also valuable. With advanced tools such as deep neural networks, it is now much easier to construct these more specialized features.

6 Conclusion

Our goal in this project is to use machine learning models to predict cross-sectional stock returns and generate corresponding trading strategies. Based on the overall evaluation across different models, we find that Boosting is the most robust model, bringing us stable positive returns. There are several important points to take away. First, we need to pay attention to the different traits of micro-cap stocks and large-cap stocks, which can undermine our trading strategy; this also affects how we construct the portfolio (equal-weighted or market-cap-weighted). Second, we need to consider the huge shift in the financial market in recent years, which means that cross-sectional strategies are not as powerful and effective as before. These macro changes suggest that we should build and train different models for different periods. Third, here we are using monthly data from 1972.01 to 2025.12, during which the whole market has changed a lot; it could be better to use more recent, higher-frequency data to train our model. In general, in this project, we have tried different models and identified the limitations of each.

References